

KeyNotes_原理_cpp扩展编译 & CUDA

1. CUDA: 编译环境 & 运行环境

在深度学习工程（Python）中，（NVIDIA）CUDA 分为两个截然不同但紧密相关的部分：**系统级 Toolkit** 和 **环境级 Runtime**。

1.1 系统级 CUDA (CUDA Toolkit) —— 仅用于编译 (Build-time)

- 定义：**完整开发包**。包括核心组件：
 - `nvcc`：编译器（厨师），负责将 `.cu` 源码转为二进制。
 - `cuda.h` 等头文件：编译依赖的“菜谱”，告诉编译器函数长什么样。
 - 静态库/存根：用于编译链接阶段。
- 用途：仅用于把 C++ 源码编译成二进制文件
- 路径：`/usr/local/cuda`
- 管理：可以多版本并存。通过 `PATH` 环境变量（如 `~/.bashrc`）/conda环境变量配置控制调用哪个版本的 `nvcc`。

1.2 虚拟环境中的 CUDA (CUDA Runtime) —— 仅用于运行 (Run-time)

- 定义：**通常随 PyTorch 安装在 conda/venv 环境中**（例如 `site-packages/torch/lib`）。
- 核心组件：动态库文件，最核心的是 `**libcudart.so`。
- 用途：仅用于程序运行。它不包含头文件和编译器，无法用来编译代码。
- PyTorch 的霸道机制**：
 - PyTorch 自带经过裁剪的 CUDA Runtime，用户无需在系统安装 Toolkit 即可运行 PyTorch 代码。
 - 运行时劫持**：PyTorch 启动时会将自己的 `lib` 路径强制塞给系统加载器。因此，你编译好的扩展在运行时，通常会链接到 PyTorch 自带的那个 `libcudart.so`，而不是系统里的那个。
- 兼容性原则**：由于“运行时劫持”系统级 **CUDA（编译用）版本** 与 **PyTorch 自带 CUDA（运行用）版本** 最好一致或 ABI 兼容，否则可能出现运行时符号未定义错误。

2. cpp 扩展编译

- 一个 Python 环境中编译好的 cpp 扩展形式如下：
 - `/home/user/anaconda3/envs/pytorch/lib/python3.10/site-packages/my_extension.cpython-310-x86_64-linux-gnu.so`
 - 就是一个 `.so` 文件，这个 `.so` 文件包含：
 - .text 段**：你的纯 C++ 源码编译后的机器码（CPU 指令）。
 - .nv_fatbin 段**：你的 **CUDA Kernel** 编译后的东西（打包在一起的 SASS 和 PTX）。
 - .dynsym 段（动态符号表）**：这就是那个“借条”。它记录了：
 - 导出符号：我自己有哪些函数可以给别人用（比如 Python 调用的接口）。
 - 导入符号：我需要找别人借哪些函数（比如 `cudaMalloc`, `printf` 等）。
- 什么是 **.so (Shared Object)**？
 - 它不仅仅是一个链接，它是**实体函数库**。
 - 内容**：它包含了你编写的所有 C++/CUDA 逻辑的**完整机器码**（CPU 指令 + GPU SASS/PTX）。
 - 依赖**：它不自带外部基础库（如 CUDA Runtime），而是通过**动态链接**（留一个“借条”）的方式，在运行时去系统中借用 `libcudart.so` 来执行基础操作（如 `cudaMalloc`）。
- 编译好的 cpp 扩展（**.so + “动态符号表”**）：本身不自带 **CUDA Runtime**，它是动态链接的。要借助 `pytorch` 运行
- PTX vs SASS**：

- **PTX**: 虚拟汇编（通用）。显卡看不懂，运行时需要驱动 JIT 编译（启动稍慢，兼容性好，但运行速度无区别）。
- **SASS**: 真实机器码（特定架构）。显卡直接执行（启动快，架构不匹配则无法运行）。
- 最佳实践：通常编译结果会同时打包特定架构的 **SASS**（如 **sm_80**）和高版本的 **PTX**（用于未来显卡保底）。
- 编译工具链: **setup.py**
 - 角色: Python 调用 C++ 编译工具链的总指挥。
 - 核心参数:
 - **include_dirs**: 告诉编译器去哪找头文件。
 - **extra_compile_args**: 指定 **nvcc** 标志（如架构 **sm_xx**）。
 - 架构自动检测:
 - 若未指定架构, **torch.utils.cpp_extension** 会根据当前机器插的显卡自动设置编译参数。
 - 可通过环境变量 **TORCH_CUDA_ARCH_LIST** 手动覆盖（例如打包分发给不同机器使用时）。

2.1 Phase 1 编译全流程 (Build Pipeline)

阶段 1: 编译 (Compilation) —— 生成 .o

- 输入: 两部分源码 (.cpp, .cu) + 头文件 (.h)。
 - .cpp: 纯 C++ 部分 -> 编译成 x86_64 CPU 指令;
 - .cu: CUDA Kernel (**global** 函数) -> 编译成 SASS/PTX
- 动作:
 - 编译器 (GCC 处理 cpp, NVCC 处理 cu) 读取源码。
 - 遇到 **#include <cuda_runtime.h>** 等, 通过 **setup.py** 中的 **include_dirs** 找到头文件 (如 **cuda_runtime.h**), 复制粘贴进来, 检查语法。
 - **setup.py** 中的 **extra_compile_args** 在此阶段指定 GPU 架构 (如 **-gencode=arch=compute_80,code=sm_80**)。
- 输出: 对象文件 (.o)。包含源码对应的机器码, 但外部函数调用 (如 **cudaMalloc**) 此时只是占位符

阶段 2: 链接 (Linking) —— 生成 .so

- 输入: 多个对象文件 (.o) + 库文件信息。
- 动作:
 - 链接器将所有 .o 拼合。
 - 依赖检查: 链接器参考系统中的 **libcudart.so**, 确认外部函数 (如 **cudaMalloc**) 存在, 并在结果中写入“动态符号表” (将来运行时, 请加载名为 **libcudart.so** 的库)。
 - 如果是静态链接 (.a), 它会把 **libcudart** 的代码直接抄进你的 .so 里; 但一般是动态链接, 只记录文件名。
- 输出: **/home/user/anaconda3/envs/pytorch/lib/python3.10/site-packages/my_extension.cpython-310-x86_64-linux-gnu.so**; 这个 .so 文件包含:
 1. **.text** 段: 你的纯 C++ 源码编译后的机器码 (CPU 指令)。
 2. **.nv_fatbin** 段: 你的 CUDA Kernel 编译后的东西 (打包在一起的 SASS 和 PTX)。
 3. **.dynsym** 段 (动态符号表): 这就是那个“借条”。它记录了:
 - 导出符号: 我自己有哪些函数可以给别人用 (比如 Python 调用的接口)。
 - 导入符号: 我需要找别人借哪些函数 (比如 **cudaMalloc**, **printf** 等)。
- 最终扩展文件 (**my_extension.so**) + 一个“动态符号表” (记录了它依赖谁)
 - 能不能把源码中的 .cu 编译成 **sm_120** (**5060 Ti** 唯一对应的版本) —— 仅取决于系统级 **CUDA Toolkit** 的 **nvcc** 版本。编译器不认识的架构无法生成对应的 **SASS**, 与源码无关。

2.2 Phase 2 运行全流程 (Runtime Pipeline)

1. 导入: Python 执行 **import my_extension**。
2. 加载实体: 操作系统加载器打开 .so, 将你编写的逻辑代码 (二进制) 读入内存。
3. 解析依赖: 加载器扫描“动态符号表”, 发现依赖 **libcudart.so** (例如要调用 **cudaMalloc** 函数时)。

4. 寻路 (Resolution):

- 加载器按照 `RPATH -> LD_LIBRARY_PATH` 等顺序查找。
- **PyTorch** 介入：由于 **PyTorch** 已在内存中加载了它自带的 `libcudart.so`，加载器通常会直接将扩展的接口“插”在 **PyTorch** 的 **Runtime** 库上。(这也是为什么编译时 `cuda toolkit` 和运行时 `cuda runtime` 版本要兼容的原因。)

5. GPU 代码执行:

- 如果 `.so` 中有匹配当前显卡的 **SASS**，直接运行。
- 如果没有匹配的 **SASS** 但有 **PTX**，驱动触发 **JIT** 编译生成 **SASS** 后运行。